Reimplementing DoRA: Weight-Decomposed Low-Rank Adaptation

CS 4782 Final Project Report

Eric Do (ed547) Levi Kronenthaler (ldk67) Eli Furgeson (ehf38)

1 Introduction

Parameter-efficient fine-tuning (PEFT) makes it feasible to adapt large language models on commodity hardware, but a persistent accuracy gap separates PEFT from full fine-tuning (FT). **DoRA: Weight-Decomposed Low-Rank Adaptation** [1], by Liu, Wang, Yin, Molchanov, Wang, Cheng, and Chen (ICML 2024), is a drop-in replacement for LoRA [2] that closes much of that gap with no inference overhead.

The paper's core insight is that an FT update can be decomposed into a *magnitude* (per-output-channel scalar) and a *direction* (unit-norm vector), and that LoRA's update couples these in a way that diverges from FT. DoRA reparameterizes each pretrained linear weight as $W = m \cdot V/\|V\|_c$, learns the direction V with a low-rank update, and trains m directly. Across NLU, commonsense reasoning, and visual instruction tuning, the authors report that DoRA matches or beats LoRA, often at half the trainable-parameter budget, and merges back into a plain linear at inference time.

2 Chosen Result

We target the commonsense-reasoning portion of Table 1 of the paper: zero-shot accuracy on eight tasks (BoolQ, PIQA, SIQA, HellaSwag, WinoGrande, ARC-e, ARC-c, OBQA) for Llama-2-7B [3] and Llama-3-8B [4], comparing LoRA and DoRA across a range of ranks. We focus on the paper's two headline claims: *(i)* DoRA at rank r matches LoRA at rank $2r$, and *(ii)* DoRA's accuracy is more rank-robust than LoRA's. Both claims are reproducible on a single Colab A100, and together they constitute the most-cited evidence that weight-decomposition is doing the work, rather than just adding capacity.

3 Methodology

Implementation. `DoRALayer` (`code/dora.py`) is an `nn.Module` wrapping an `nn.Linear`, holding (i) frozen base weights W, (ii) trainable LoRA factors $A \in \mathbb{R}^{r\times d_{\text{in}}}$ (Kaiming-init) and $B \in \mathbb{R}^{d_{\text{out}}\times r}$ (zero-init), and (iii) a trainable magnitude vector m initialized to the row-wise ℓ_2 norm of W. The forward pass is

$$W' = m \cdot \frac{W+\frac{\alpha}{r}BA}{\left\|W+\frac{\alpha}{r}BA\right\|_{\text{row}}+\varepsilon}, \quad y = W'x + b.$$

At step 0, $B{=}0$ and $m{=}\|W\|$, so $W'{=}W$ exactly, the layer is identity-initialized. A `merge_and_unload` routine bakes W' back into the underlying linear at inference time, so DoRA carries *zero* latency overhead post-training. LoRA is implemented via the HuggingFace PEFT library [5] for an apples-to-apples baseline, using the same target modules, rank, and α.

BiDoRA extension. As a small extension beyond the main DoRA reproduction, we implemented a BiDoRA-inspired training mode [6]. BiDoRA argues that DoRA's simultaneous optimization of magnitude m and direction ΔV can over couple the two components, so it decouples them with a bi-level objective. Direction is learned on a training split while magnitude is updated using validation feedback. Our version is a lightweight approximation of this idea, we simply alternate optimizer steps between direction-only updates (A, B) and magnitude-only updates (m), which lets us test whether decoupling the two DoRA components changes training behavior.

Targets and hyperparameters. Following the paper we apply LoRA/DoRA to the attention projections `q_proj` and `v_proj` only, with $\alpha = 2r$. We sweep both methods at $r \in \{4, 8, 16, 32, 64\}$ on both Llama-2-7B and Llama-3-8B (20 runs total). Training uses batch 4 with gradient accumulation 4 (effective 16), bf16, lr 2×10^{-4} with cosine schedule and 50-step warmup, on the Commonsense-170K instruction set [7].

Deviations from the paper. Two compute-driven simplifications: (a) we cap training at **1,000 optimizer steps** ($\sim$16K examples; about $1/10$ of one epoch) rather than the paper's 3 epochs, and (b) we restrict adapter targets to $\{q, v\}$ rather than the paper's $\{q, k, v, \text{up}, \text{down}\}$. We also use a single learning rate across all trainable parameters, whereas the paper uses a separate (typically $2\times$) learning rate for m versus BA.

Evaluation. After training we run `merge_and_unload` and evaluate the resulting plain HuggingFace model zero-shot via `lm-eval-harness` [8] on seven of the paper's eight tasks (we omit SIQA due to a harness configuration mismatch). The identical protocol is applied to the merged-LoRA baselines.

Table 1 reports per-task accuracy and the unweighted mean across the seven tasks for all 20 runs.

Base model	Method (rank)	BoolQ	PIQA	HellaSwag	WinoGrande	ARC-e	ARC-c	OBQA	**Avg**
	LoRA r=4	78.50	79.05	75.70	71.82	69.91	40.87	43.60	**65.64**
	LoRA r=8	79.82	79.49	76.15	71.74	73.53	42.66	43.00	**66.63**
	DoRA r=4	77.74	79.05	75.55	72.38	70.71	41.89	44.00	**65.90**
	DoRA r=8	79.94	79.11	76.40	72.22	71.72	42.41	43.60	**66.49**
Llama-2-7B	LoRA r=4	82.75	81.45	78.77	74.11	76.98	50.77	44.60	**69.92**
	LoRA r=8	82.60	81.45	78.63	75.22	78.83	52.47	43.60	**70.40**
	DoRA r=4	83.39	81.56	78.63	76.48	80.18	54.01	43.40	**71.09**
	DoRA r=8	83.52	82.10	78.57	76.24	79.04	53.33	44.40	**71.03**
Llama-3-8B									

Table 1: Zero-shot accuracy (%) per task for low ranks $r \in \{4, 8\}$. PIQA, HellaSwag, ARC-e, ARC-c, and OBQA use `acc_norm`; BoolQ and WinoGrande use `acc`, following the harness defaults.

4 Results & Analysis

Headline claim reproduced on Llama-3. At low rank, DoRA outperforms LoRA on Llama-3-8B by margins consistent with the paper. At r=4, DoRA scores 71.09 vs LoRA's 69.92 (+1.17 **pts**); at r=8, 71.03 vs 70.40 (+0.63). The gap narrows at r=16 and reverses at r=32/r=64, where LoRA narrowly leads. This matches the paper's framing: the decomposition is most useful when capacity is scarce; once rank is large enough, plain LoRA can recover the same accuracy.

Half-rank claim reproduced on Llama-3. The paper's signature claim is that DoRA at rank r matches LoRA at rank $2r$. We see exactly this pattern: DoRA r=4 (71.09) *beats* LoRA r=8 (70.40) by 0.69; DoRA r=8 (71.03) is within 0.14 of LoRA r=16 (71.17); DoRA r=16 (70.62) is within 0.33 of LoRA r=32 (70.95). DoRA achieves the same accuracy with half the trainable parameters at every comparison point.

Rank robustness is the cleanest signal. Across $r \in \{4, 8, 16, 32, 64\}$, DoRA's average accuracy spreads only **0.47 pts** on Llama-3 and **0.96 pts** on Llama-2. LoRA spreads **1.25 pts** and **1.69 pts** respectively. DoRA is essentially flat across a 16× range of ranks on Llama-3 (70.62–71.09); LoRA needs $r \geq 16$ to reach plateau and degrades sharply below it. This is the qualitative match to the paper's rank-robustness analysis (Fig. 4 in the original).

Llama-2 shows a muted version of the same effect. On Llama-2-7B the curve is qualitatively similar but the gap is smaller (DoRA +0.26 at r=4, −0.14 at r=8). DoRA peaks at r=16 (66.86) and dips to 66.27 at r=32 before recovering — the dip is driven by ARC-e (71.68 vs 72.98 at r=16) and is most likely seed/fine-tuning noise, not a structural artifact. We attribute the muted overall effect to undertraining: 1,000 steps is ∼10% of one epoch on Commonsense-170K, vs the paper's three full epochs. The DoRA advantage appears to scale with both model size and training duration, both of which favor the paper's regime over ours.

No instability or merge bugs. Across all 20 runs we observed no NaNs, divergence, or merge-time numerical drift; merged-vs-unmerged forward passes agreed to bf16 tolerance, including at the smallest rank (r=4) where LoRA's update is most aggressively scaled.

5 Reflections

Lessons learned. The paper's claims are real but rank-dependent. The DoRA > LoRA gap is not a uniform 1-point lift: it lives almost entirely at low rank ($r \leq 8$) and on larger base models. This is consistent with the paper's framing. **Rank robustness is the most stable signal.** Even where per-rank gaps were noisy, the spread across ranks consistently favored DoRA on both models. This suggests rank robustness, rather than absolute accuracy at a given rank, is the cleanest empirical fingerprint of the decomposition. **Identity initialization is a useful sanity check.** B=0, m=$\|W\|$ must reproduce the base model exactly at step 0; we caught one ε-placement bug this way. **The mathematical core fits in ∼15 lines of PyTorch.** Almost all of `train.py` is logistics (target discovery, state-dict serialization, merge semantics, optional Hub upload).

What we would do next. Train at least one config for the full 3 epochs to confirm the DoRA gap widens at scale and to test whether the Llama-2 muted-effect hypothesis (undertraining) is correct. Extend targets to $\{q, k, v, \text{up}, \text{down}\}$ and split the learning rate so m trains at a different rate than BA, the two methodological deviations remaining from the paper's recipe.

Future directions. The magnitude/direction split is general, applying it to other PEFT families (VeRA, IA^3), to convolutional weights, or to diffusion-model adaptation are natural extensions. The paper itself shows the decomposition composes with quantization and with VeRA[1].

References

[1] S.-Y. Liu, C.-Y. Wang, H. Yin, P. Molchanov, Y.-C. F. Wang, K.-T. Cheng, and M.-H. Chen. DoRA: Weight-Decomposed Low-Rank Adaptation. *ICML*, 2024.

[2] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen. LoRA: Low-Rank Adaptation of Large Language Models. *ICLR*, 2022.

[3] H. Touvron et al. Llama 2: Open Foundation and Fine-Tuned Chat Models. arXiv:2307.09288, 2023.

[4] AI@Meta. The Llama 3 Herd of Models. 2024.

[5] S. Mangrulkar, S. Gugger, L. Debut, Y. Belkada, S. Paul, and B. Bossan. PEFT: State-of-the-art Parameter-Efficient Fine-Tuning. `https://github.com/huggingface/peft`, 2022.

[6] P. Qin, R. Zhang, and P. Xie. BiDoRA: Bi-level Optimization-Based Weight-Decomposed Low-Rank Adaptation. arXiv:2410.09758, 2024.

[7] Z. Hu, L. Wang, Y. Lan, W. Xu, E.-P. Lim, L. Bing, X. Xu, S. Poria, and R. K.-W. Lee. LLM-Adapters: An Adapter Family for Parameter-Efficient Fine-Tuning of Large Language Models. *EMNLP*, 2023.

[8] L. Gao et al. A framework for few-shot language model evaluation. `https://github.com/EleutherAI/lm-evaluation-harness`, 2023.